

ShrimpFlow: 程序员的行为记忆与协作驱动系统

AudaxLab

中关村北纬龙虾大赛 · 生产力龙虾赛道

摘要

程序员每天与 AI 编程助手协作时会产生大量行为数据，包括终端操作、代码提交、对话记录和调试过程，但这些数据在对话结束后即被丢弃。AI 无法积累对用户习惯的了解，每次协作都是一次冷启动。ShrimpFlow 在开发者与 OpenClaw 之间构建了一层持久的行为记忆。系统分为四层：Shadow 层静默采集终端、Git、OpenClaw 对话等多源行为数据；Mirror 层将原始数据转化为可视化的开发者画像；Brain 层通过语义归纳与规则匹配提取稳定的个人行为模式，构建结构化的行为模型 ClawProfile；Autopilot 层将 ClawProfile 注入 OpenClaw 的协作上下文，使其按照开发者本人的习惯执行任务。在 OmniStack 机器人全栈系统的开发中，AI Agent 基于 ClawProfile 独立完成了 32 轮迭代，产出 118 次代码提交、16,824 行核心代码和 704 个测试用例。提交格式匹配率从早期的 0% 收敛至后期的 100%，全程零人工代码修改。ClawProfile 支持打包与分享，使个人开发经验成为可传递、可组合的结构化资产。

Date: March 2026

Correspondence: liyufeng854@gmail.com

1 问题背景

假设你用 OpenClaw 写了一下午代码。它帮你重构了一个模块，调通了一个并发 bug，还按你的习惯把 PR 拆成了三个小提交。协作很顺畅。然而第二天你再打开它，问一个类似的问题，它完全不记得昨天发生过什么。

这不是 OpenClaw 独有的问题。当前所有 AI 编程助手都面临同一个限制：对话结束后，上下文随即消失。AI 不记得你昨天解决过什么问题，不了解你偏好的代码风格，也不知道你在过去几个月里积累了哪些技术判断。每一次协作都从零开始。

与此同时，开发者每天产生大量行为数据。终端里执行的命令、Git 上提交的代码、与 AI 的对话记录、调试时的排查路径，这些数据散落在不同工具中，以原始日志的形式存在，始终没有被系统性地利用过。WakaTime 等工具只统计编码时长和语言分布，无法捕捉行为层面的信息。例如，开发者遇到问题时习惯先查日志还是先加断点，做架构选型时更看重社区活跃度还是 API 设计质量，这些信息在现有工具中都不会被记录。

这两个问题指向同一个缺口：程序员和 AI 之间缺少一层持久的行为记忆。当前 AI 编程工具的竞争集中在代码生成质量上，但一个更基本的问题尚未解决：再好的代码生成能力，如果不了解使用者的习惯和偏好，协作效率就会受限于反复的冷启动。开发者需要在每次会话中重复解释自己的偏好，AI 则在每次会话中重新猜测用户的意图，双方不断在信息对齐上消耗精力。ShrimpFlow 正是为了

填补这一层。它不做代码生成，不替代任何现有工具，只专注一件事：让 **AI** 记住你是怎么工作的，并据此提供个性化的协作。

2 现有方案与不足

目前业界对 **AI** 编程助手记忆缺失问题的尝试大致分为三类。

第一类是对话历史持久化。部分工具允许保存历史对话记录，在新会话中检索过去的聊天内容。然而对话记录本身是非结构化的自然语言，检索质量受限于关键词匹配或向量相似度，无法从中提取出“这个人习惯怎么做”这样的行为信息。一位开发者可能在过去三个月的对话中多次提到偏好函数式风格，但第四个月提问时如果没有再次说明，**AI** 仍不会主动参照这一偏好。

第二类是规则文件方案。部分工具允许用户手动编写一份规则文件放在项目目录下，**AI** 在每次会话开始时读取该文件作为参考。这种方式的问题在于，它将记忆的负担完全交给了用户：开发者需要自己意识到哪些习惯值得记录，再用准确的语言将其表达为规则。实际使用中大多数人只会写几条粗略的指令，覆盖不了真实的工作方式。而且规则文件是静态的，不会随着习惯的变化自动更新。

第三类是开发数据统计工具。**WakaTime** 等产品能统计编码时长、语言分布等数值指标，但停留在“做了多少”的层面，进不到“怎么做”的层面，也无法将统计结果转化为可驱动 **AI** 行为的模型。

ShrimpFlow 与这三类方案的区别在于切入角度不同。对话历史方案记住的是“你说过什么”，规则文件方案记住的是“你写下了什么”，**ShrimpFlow** 记住的是“你实际怎么做的”。它从真实的操作行为出发，自动完成从观察到归纳再到驱动的完整闭环。开发者不需要手动总结自己的习惯，也不需要维护任何配置文件，系统会从日常工作中自动学习并持续更新。

3 系统设计

ShrimpFlow 位于开发者和 **OpenClaw** 之间，负责采集行为数据、提取行为模式、驱动个性化协作。系统分为四个层次，各层之间依次递进。

层次	名称	功能
第一层	Shadow	在后台静默采集终端命令、Git 操作、 OpenClaw 对话和环境变化，不干扰开发流程
第二层	Mirror	将原始数据转化为可视化的时间线、技能图谱和日报，帮助开发者回顾自己的工作方式
第三层	Brain	从历史数据中提取稳定的行为模式，包括调试路径、编码偏好和工作节奏，构建 ClawProfile
第四层	Autopilot	将 ClawProfile 注入 OpenClaw 的上下文，使其按照开发者的习惯执行任务

Shadow 层的采集完全在后台运行，开发者无需做任何额外操作。终端命令通过 **zsh** 的 **precmd/pre-exec** 钩子捕获，连同工作目录、退出码和执行耗时一起写入数据库。**Git** 操作通过 **post-commit** 和 **post-checkout** 钩子采集，记录提交信息、**diff** 摘要和分支切换。**OpenClaw** 的对话通过会话接口获取，记录用户的提问、**Agent** 的执行动作以及用户对结果的后续修改。这里“用户对结果的后续修改”特别重要：如果 **Agent** 生成了一段代码后用户手动修改了其中几行，这个修改行为本身就携带了关于用户偏好的信息，**Shadow** 层会将原始输出和修改后的版本一起记录，为 **Brain** 层的模式提取提供对比素材。系统还定时采集运行环境快照，包括活跃进程和依赖变更。所有数据写入本地 **SQLite** 数据库，不上传任何外部服务器。采集对开发体验几乎没有影响，所有 **hook** 以异步方式写入，实测中对终端命令执行的延迟增加不超过 2 毫秒。

Mirror 层将原始事件组织为多个粒度的开发者画像。时间线按事件类型着色，呈现编码、调试和 **AI** 对话在各时段的分布，开发者可以一目了然地看到自己的时间花在了哪里。技能图谱统计各项

技术的使用频率，并追踪其随时间的变化趋势，例如某段时间内 PyTorch 使用频率上升而前端技术下降，意味着工作重心向模型训练方向转移。日报由 LLM 根据当天事件自动生成结构化摘要，记录完成了什么、遇到了什么问题、采取了什么解决方式。周报则汇总更长时间跨度上的趋势性变化，帮助开发者从更宏观的视角审视自己的工作方式。Mirror 层的输出本身就具有实用价值：即使不使用后续的 Brain 和 Autopilot 层，开发者也能通过 Mirror 层获得对自身工作方式的系统性认识。

Brain 层是系统的核心，负责将原始行为数据提炼为可复用的行为模式。提取过程分两步进行。第一步由 LLM 对一段时间内的事件序列做语义归纳，识别其中重复出现的操作模式，例如“每次新建模块前先写测试骨架”或“提交前总会运行一遍完整的测试套件”。第二步通过规则匹配验证这些模式的稳定性：只有在多次独立观察中一致出现的模式才会被纳入 ClawProfile，偶发的行为不会被误判为稳定模式。提取的模式覆盖多个维度，包括工作流（分支策略、提交节奏、PR 粒度）、编码偏好（命名惯例、异常处理方式）、调试策略（排查路径、日志使用偏好）和时间节奏（高效工作时段、任务切换频率）。每条模式附带一个置信度分数和观察次数，随新数据持续更新。当某个模式长时间不再出现时，置信度会逐步衰减，最终被标记为归档状态，不再注入 Autopilot。

Autopilot 层将 ClawProfile 中的模式注入 OpenClaw 的系统提示。注入并非简单地将整个 ClawProfile 拼接进去，而是根据当前任务的类型和上下文，从中选取最相关的模式子集。例如，当用户正在做代码审查时，系统会优先注入与审查标准、关注重点相关的模式，而非提交格式或调试策略。这种选择性注入机制避免了提示过载对 Agent 推理质量的干扰。Autopilot 有三个级别：建议模式下 Agent 仅给出参考，由开发者自行决定是否采纳；半自动模式下用户给出指令后 Agent 按学到的习惯执行具体步骤；全自动模式下 Agent 接管任务队列，按用户的方式自主完成一系列工作。OmniStack 案例中迭代 16 至 32 阶段使用的就是全自动模式。三个级别之间可以随时切换，开发者始终拥有最终控制权。

4 案例研究：OmniStack

为验证 ShrimpFlow 的实际效果，我们将其应用于 OmniStack 的完整开发过程。选择 OmniStack 作为验证对象是因为它的规模和复杂度足以检验 ShrimpFlow 在真实工程场景中的表现。OmniStack 是一个面向机器人领域的全栈研发系统，覆盖从数据采集到策略部署的完整闭环。系统包含四条管线：数据管线负责采集、格式转换、质量审计和多数数据集混合；策略训练管线支持 Diffusion Policy、Flow Matching、iMLE 和 VLA 四种路线；策略评估管线对接 MuJoCo 和 LIBERO 仿真环境；策略部署管线包含推理服务、WebSocket API 和 Sim2Real 域随机化。技术栈涵盖 PyTorch、PyTorch Lightning、HuggingFace Transformers 和 MuJoCo 等组件。整个项目在约 46 小时内经历了 32 轮迭代，最终产出 101 个 Python 源文件（16,824 行核心代码）、704 个测试用例和 118 次 Git 提交。图 1 展示了这一过程的全景：热力图和时间线记录了 Agent 在 46 小时内的开发节奏，每轮迭代严格遵循 `feat` → `test` → `docs` 三合一节奏，提交格式和分支命名与开发者本人的习惯完全一致，全程零人工干预。以下从四个角度展示 ShrimpFlow 在这个项目中的具体表现。

4.1 开发过程可视化

在 OmniStack 的开发过程中，Mirror 层持续将 Shadow 采集到的事件流转化为可视化画像。图 2 展示了生成的开发者画像。时间线按事件类型着色，呈现了编码、调试、AI 对话和文档工作在各时段的分布。技能图谱追踪了 Python、PyTorch、MuJoCo 等技术的使用频率变化，其中策略训练相关技术在迭代 16 之后的占比明显上升。日报以结构化摘要记录了每轮迭代的工作内容和遇到的问题。这些信息此前散落在终端历史、Git 日志和聊天记录中，Mirror 层将它们第一次整合为连贯的画面。

4.2 行为模式的收敛过程

OmniStack 的 Git 历史完整记录了 Agent 行为从无序到规范的收敛过程。在迭代 8 至 12 阶段，Brain 层尚未完成模式提取，Agent 的提交格式不稳定，表现为 `feat: iter-9 --- data augmentation,`



Figure 1 OmniStack 全自动开发过程全景：32 轮迭代、118 次提交的热力图与时间线

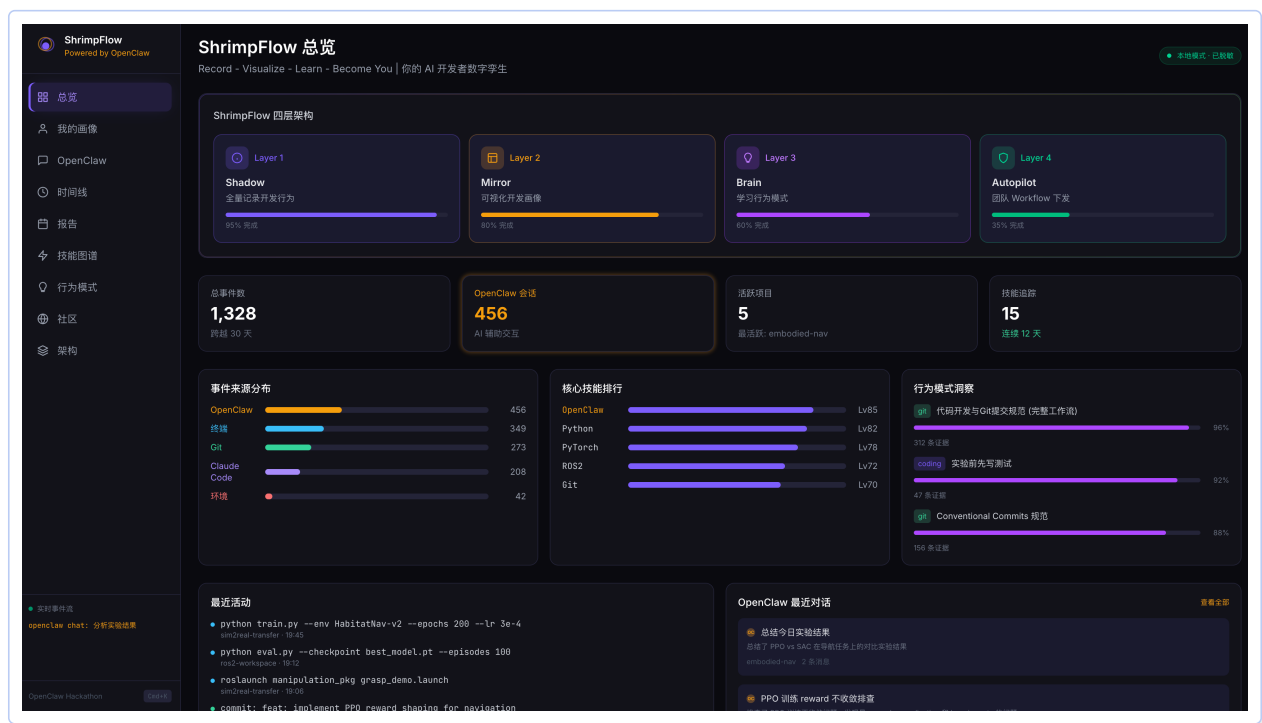


Figure 2 Mirror 层为 OmniStack 开发过程生成的开发者画像与时间线

multi-dataset mixing 这样的粗粒度描述，一个 commit 里混着整个迭代的所有变更。这 13 条提交是 118 条总提交中仅有的不符合 [scope/op]: 格式的非 Merge 提交。迭代 13 至 15 是过渡期。Brain 层开始输出初步提取的 Git 工作流模式后，Agent 学会了按模块拆分 scope，但迭代节奏还不稳定。

从迭代 16 起，行为完全收敛。此后连续 17 轮迭代共 53 条提交，全部严格遵守 [iterN/op]: description 格式，每轮固定产出 feat → test → docs 三条提交，没有任何例外。每条 commit message 附带结构化的变更说明和测试通过数。这些格式并非预设的模板，而是 Agent 从 ClawProfile 中学到的开发者习惯。

这一过程说明行为模式的提取和注入是渐进的。Brain 层不会在某一时刻一次性输出完整的行为模型，而是随观察数据的积累逐步完善。早期 Agent 因缺少行为参照而使用通用策略，中期部分模式开始生效，后期行为完全收敛。这正是 ShrimpFlow 的设计预期。

阶段	行为特征	格式匹配率
iter 8–12	粗粒度提交，功能、测试、文档混在一起	0/13 (0%)
iter 13–15	开始按模块拆分 scope，节奏尚不稳定	部分匹配
iter 16–32	严格三合一节奏，格式完全收敛	53/53 (100%)

4.3 VLA 模块的完整交付

迭代 16 之前，OmniStack 有三种策略实现 (Diffusion Policy、Flow Matching、iMLE)，Vision-Language-Action (VLA) 模块完全不存在。下面记录 Agent 如何基于 ClawProfile 从零完成这个模块。

迭代 16 中 Agent 接到 VLA 进化目标后，创建了 omnistack/policies/vla/ 目录，实现了基于

HuggingFace Transformers 和 LoRA/PEFT 的 VLMEncoder, 以及 FlowMatching 和 L1 两种 action head。同一轮中还注册了 LIBERO 仿真环境适配器, 添加了 DDP 分布式训练支持和 SE(3) 数据增强, 共交付 6 个进化目标, 附带 52 个新增测试全部通过。迭代 17 至 20 期间, Agent 在 VLA 模块上连续迭代, 依次加入了语言条件 (language conditioning)、时间集成推理 (temporal ensemble)、交叉注意力 action head、KV cache 和 RoPE 位置编码。每一轮都遵守 feat → test → docs 的节奏。

迭代 22 中 Agent 发现 LoRA 的 peft_config 在保存和加载 checkpoint 时会丢失, 随即自行修复并添加了完整的序列化支持。迭代 25 中 Agent 运行测试时遇到 9 个 VLA 相关测试失败, 自主定位到 batched normalizer 的读取逻辑和 checkpoint 格式兼容性两个原因, 一次性修复了全部失败。最终 VLA 模块产出 1,216 行核心代码, 支持三种 action head、完整的 checkpoint 序列化、LoRA 微调、RoPE 位置编码和 KV cache 推理加速。全过程 Agent 独立完成, 开发者只参与方向确认。

这个案例值得关注的地方在于 Agent 不是在执行预写好的脚本。ClawProfile 中存储的是从行为数据归纳出的模式, Agent 据此自主决定何时建分支、如何拆 commit、PR 粒度多大、测试如何组织。遇到 ClawProfile 未覆盖的情况时, Agent 在遵守整体工作流的前提下灵活处理了具体问题。例如, 迭代 22 中 Agent 发现 LoRA checkpoint 的序列化缺陷后, 是在保持 [iter22/feat] 提交格式的同时, 自主设计了修复方案并补充了对应的测试, 而不是等待人工指导。

更值得注意的是 Agent 在后期表现出的主动维护行为。迭代 31 中 Agent 识别出项目里累积的 38 个 lint 错误, 自行发起了一轮技术债务清理, 格式化了全部 100 个源文件。迭代 32 中 Agent 判断按迭代编号组织的 15 个测试文件已经不利于维护, 于是自主将其合并为 6 个按功能域组织的文件。这两轮迭代不是由开发者发起的, 而是 Agent 在评估项目状态后自行决定执行的。这些行为说明 Agent 不只是在机械地遵守学到的模式, 也在模式的框架内展现出了工程判断能力。

4.4 量化结果

指标	数值	说明
总提交数	118	含 5 条 Merge, 其余均为 Agent 独立产出
格式匹配率	84.1%	排除 Merge 后的比例; 迭代 16 之后为 100%
PR / Merge	6	3 次 PR、2 次 dev→main 合并、1 次 feature 合并
核心代码	16,824 行	101 个 Python 文件, 13 个功能模块
测试用例	704	从迭代 16 的 449 个增长至迭代 32 的 704 个
代码净增	+45,000 行	插入 142,493 行, 删除 97,398 行
开发周期	约 46 小时	32 轮迭代, 平均 2.6 次提交/小时
人工代码修改	0 次	开发者仅参与方向确认

图 3 展示了 Brain 层从 OmniStack 开发历史中提取的 Git 工作流模式, 图 4 展示了迭代 16 至 32 阶段的提交记录, 可以直观看到三合一的提交节奏。

5 ClawProfile 与经验流通

ShrimpFlow 的核心产物是 ClawProfile, 一个从开发者行为中提取的结构化行为模型。ClawProfile 不是静态文件, 而是随着新数据的写入持续更新的活档案。每个条目包含四部分内容: 模式描述、适用场景、证据链和观察统计。开发者可以浏览自己的 ClawProfile, 删除不准确的条目, 补充 Brain 层尚未自动识别的模式, 或调整条目的优先级。

软件行业长期面临隐性知识传递的难题。调试时的判断、架构选型时的权衡、代码审查时的侧重,



Figure 3 Brain 层从 OmniStack 开发历史中提取的 Git 工作流模式

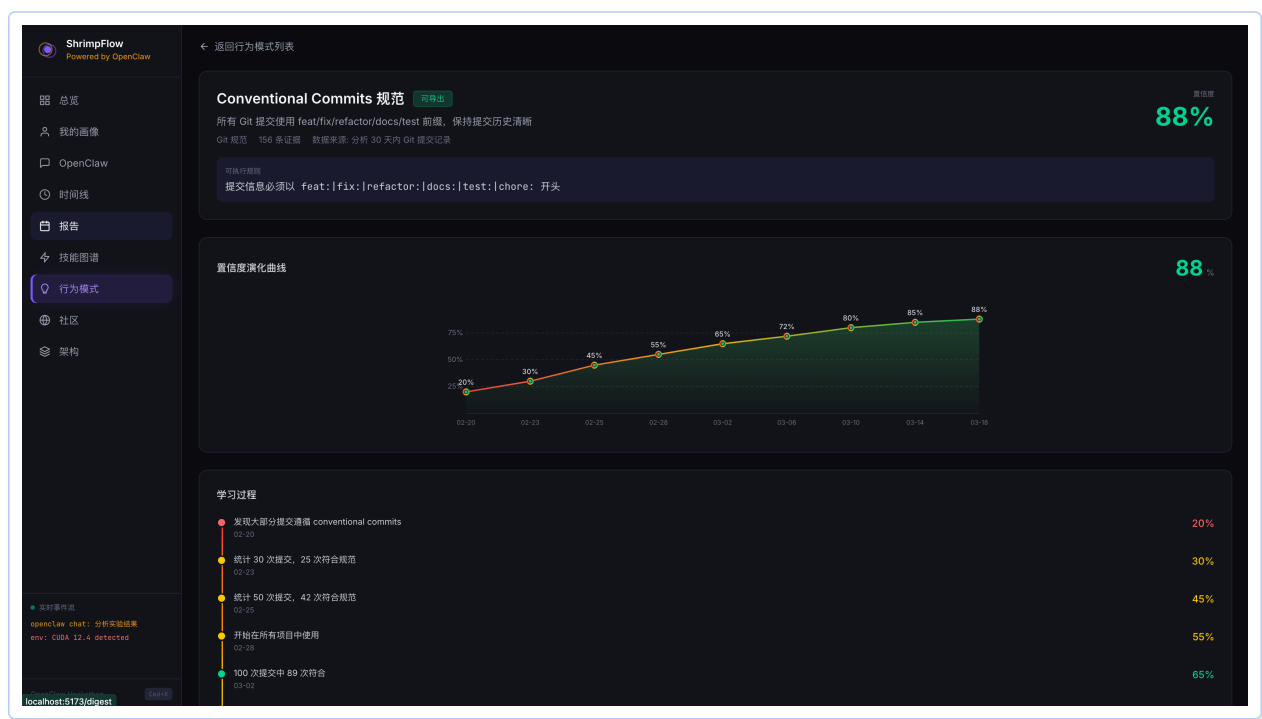


Figure 4 OmniStack 迭代 16 至 32 阶段的提交记录, 每轮固定三条提交

这些能力构成了工程师的核心竞争力，但很难通过文档或培训传递给他人。一位资深工程师离开团队后，他多年积累的工作方法往往随之流失。**ClawProfile** 提供了一条新的传递路径。

传统方式	局限	ClawProfile 的做法
技术博客	传递的是知识而非行为模式	直接从行为中提取模式本身
代码规范文档	只覆盖显式规则	从实际操作中归纳，包含隐性偏好
结对编程	一对一，无法规模化	一次建模，多人可导入
阅读源码	只能看到结果	记录的是决策过程本身

ClawProfile 在实践中有三种典型用法。在团队经验传承场景中，资深成员分享自己的 **ClawProfile** 后，新人导入即可让 **OpenClaw** 按照团队的代码规范、分支策略和测试标准工作。以 **OmniStack** 为例，如果新加入的开发者导入了这份 **ClawProfile**，**OpenClaw** 会自动按照 **[scope/op]** 的格式提交代码、按 **feat** → **test** → **docs** 的节奏组织每轮迭代，不需要新人逐条阅读规范文档，也不需要反复试错中摸索惯例。在模式组合场景中，**ClawProfile** 的模块化结构允许从多个来源导入不同维度的模式并混合使用，比如采用一位同事的架构判断搭配另一位测试策略，组合后的 **Profile** 会在使用中持续进化。在社区流通场景中，开发者可以将 **ClawProfile** 发布到开放平台，优秀的行为模式像开源代码一样自由传播和复用。在 **ShrimpFlow** 的设想中，每位开发者的 **OpenClaw** 都可以通过导入优秀的 **ClawProfile** 获得增强，从一只小龙虾成长为一只大龙虾。

6 与 OpenClaw 的关系

ShrimpFlow 不替代 **OpenClaw**，而是为它补上记忆。可以把两者的关系理解为长期记忆和工作记忆的配合：**OpenClaw** 在每次会话中有很强的即时处理能力，**ShrimpFlow** 负责在会话之间保存和积累用户的行为模型。

OpenClaw 本身是一个功能完备的 AI 智能体框架，支持本地运行和系统级任务执行，能够自主操作文件、执行代码、管理开发流程。它的局限不在能力，而在记忆：每次会话结束后对用户的了解归零，下一次协作必须从通用状态重新开始。

ShrimpFlow 补上了缺失的这一环。**Shadow** 层持续记录用户与 **OpenClaw** 的每一轮交互：用户提出了什么需求、**Agent** 执行了什么操作、用户对结果做了哪些修改。**Brain** 层从这些交互中学习用户的偏好，包括代码风格、PR 拆分方式、错误处理策略和文档详细程度等。**Autopilot** 层将学到的偏好注回 workflow，使 **OpenClaw** 在下次协作时直接按用户的方式工作。随着协作次数增加，**OpenClaw** 对用户的了解逐步加深：使用第一周了解基本编码风格，一个月后掌握分支策略和调试偏好，更长时间后对不同类型任务的决策倾向形成系统性认知。**OmniStack** 中观察到的收敛过程就是这一闭环的实际缩影。

从技术集成角度看，**ShrimpFlow** 对 **OpenClaw** 是单向依赖、零侵入的。数据读取走的是 **OpenClaw** 已有的会话接口，模式注入走的是系统提示，不需要修改 **OpenClaw** 的内部实现。**ShrimpFlow** 可以作为独立模块部署，当 **OpenClaw** 版本更新时也能无缝适配。**OpenClaw** 提供执行能力，**ShrimpFlow** 提供对用户的了解，两者结合后 **OpenClaw** 才能从通用工具变成了解用户的协作伙伴。

7 安全设计

ShrimpFlow 处理的是个人开发行为数据，安全与隐私是系统设计的前提。

原则	具体措施
本地存储	所有行为数据写入本地 SQLite，不经过外部服务器
自动脱敏	写入前自动识别并脱敏 API Key、密码、Token 等敏感内容
用户控制	ClawProfile 的分享范围由用户决定，默认不公开
可审计	每条模式关联完整的证据链，可随时查看、修改或删除
最小采集	仅采集开发活动相关数据，不监控其他应用

在数据流转的每一个环节都有对应的隐私保护措施。Shadow 层写入数据库前自动脱敏敏感字符串。Brain 层调用 LLM API 前会移除代码实体细节，只保留行为结构描述，例如“用户修改了 `auth_service.py` 中的 `login` 函数”会被抽象为“用户修改了某服务模块中的认证函数”。如果用户对外部调用仍有顾虑，可以选择本地部署的模型。分享 ClawProfile 时系统还会执行二次脱敏检查，确保输出内容中不含个人身份信息或项目机密。整个安全设计的思路是在每一层设置独立的保护机制，层层兜底。

8 技术实现

模块	技术选型	选型理由
Shadow	zsh / git / OpenClaw 钩子	利用已有 hook 机制，零侵入，异步写入
存储	SQLite (WAL 模式)	单文件部署，并发安全，适合本地场景
Brain	LLM API + 规则引擎	语义归纳发现模式，规则引擎验证稳定性
Mirror	Vue 3 + D3.js + Three.js	2D/3D 可视化，组件化开发
Autopilot	Prompt injection	将 ClawProfile 注入 OpenClaw 的系统提示

所有采集源产出统一格式的事件，写入同一张表。每条事件包含来源标识、事件类型、时间戳、上下文（工作目录、所属项目、当前分支）和事件负载。统一的数据模型是整个系统的基石，它使 Brain 层可以跨数据源识别模式，比如发现“每次修改配置文件后都会立即运行测试”这种跨终端和 Git 的关联行为，或者“下午三点到五点的代码提交质量明显高于晚间”这种跨时段的效率规律。Shadow 层采用插件式架构，每个采集源作为独立插件注册，新增采集源只需实现统一的事件接口即可接入。当前已支持终端、Git 和 OpenClaw 对话三类数据源，后续计划扩展到 IDE 内的编辑行为（如光标移动轨迹、代码补全接受率）和 CI/CD 流水线的执行结果。图 5 展示了各模块之间的数据流向。

9 总结与展望

ShrimpFlow 针对的是一个具体问题：AI 编程助手缺少对用户的持久记忆。系统通过静默采集、行为建模和个性化驱动三个环节，将散落的开发行为数据转化为结构化的行为模型，再注入 OpenClaw 的协作流程。OmniStack 案例验证了这一方法的可行性：格式匹配率从 0% 逐步收敛至 100%；Agent 独立完成了 118 次提交、16,824 行代码、704 个测试，全程零人工代码修改；Agent 还展现了模式之外的工程判断力，包括自主修复 bug、清理技术债务和重组测试结构。

ClawProfile 使个人开发经验成为可传递、可组合的资产。将行为模式结构化之后，知识的传递变成了文件的导入，接收方不需要理解模式背后的全部逻辑，导入即可使用。当越来越多的开发者在社区中分享自己的 ClawProfile 时，优秀的开发经验将不再局限于个别团队，而是成为整个社区的公共资源。

未来工作将沿三个方向推进。

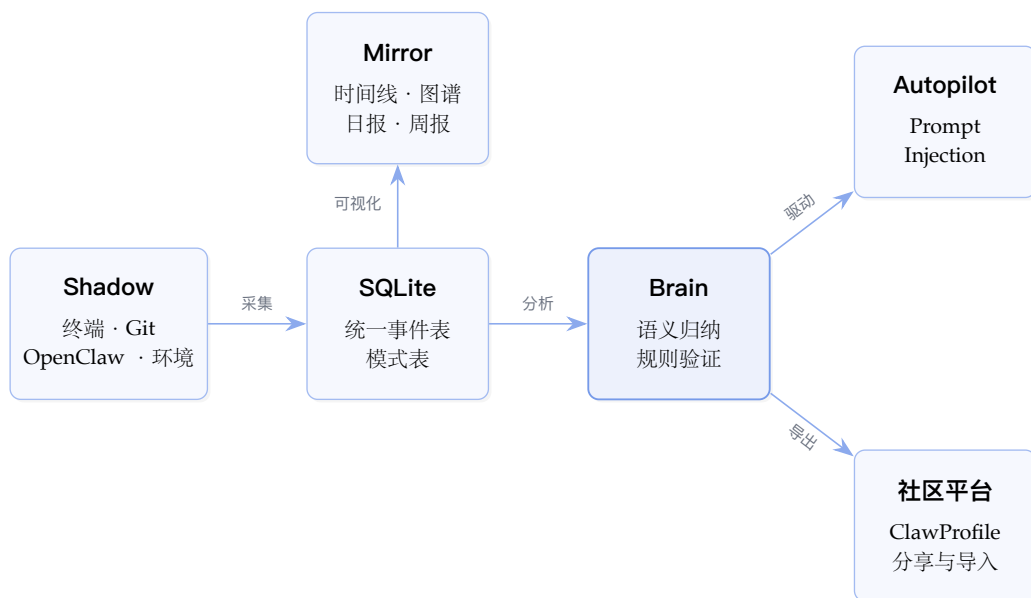


Figure 5 ShrimpFlow 系统架构与数据流向

在模式提取方面，当前的语义归纳依赖通用 LLM，对高度专业化的开发场景（例如分布式系统调优、安全漏洞排查）的识别还不够精细。后续计划在开发行为语料上做专项微调，提升跨工具、跨时段的复杂行为关联识别能力。同时引入多轮确认机制，当新识别的模式与已有模式存在矛盾时主动请求用户确认，而非自动覆盖。OmniStack 案例中迭代 8 至 15 的收敛延迟也暴露了冷启动阶段的不足，后续将允许用户在首次使用时导入一份基础 ClawProfile 作为初始种子，缩短从零到可用的过渡期。

在社区建设方面，需要建立 ClawProfile 的版本管理、质量评价和兼容性检测机制，让开发者可以像管理代码仓库一样对行为模型进行 fork 和 merge。平台还将提供基于技术栈和工作内容的匹配推荐，帮助开发者更快找到对自己有用的 Profile。社区治理同样重要：如何识别和淘汰低质量的 ClawProfile，如何激励高质量 Profile 的贡献者，这些问题需要通过社区实践逐步建立规则。

在团队建模方面，当前 ClawProfile 聚焦于个人开发者，但软件开发本质上是团队协作。后续计划从个人 ClawProfile 扩展到团队级别的行为模型，捕捉协作中的分工模式、代码审查节奏和技术决策流程。团队行为模型的核心挑战在于如何处理个人习惯与团队规范之间的张力。一个值得探索的方向是多位成员 ClawProfile 之间的自动协调：当不同成员的模式在同一维度上存在差异（例如不同的 commit message 风格或不同的 PR 审查标准）时，系统能识别出这些差异，呈现给团队讨论，并在达成共识后将统一的规范写入团队级别的 ClawProfile。这样 AI 不仅了解每个人的工作方式，也能理解团队整体的运作逻辑，在协作场景中提供更精准的支持。